

Los Evita Bloqueos

Proyecto de consultoría de rendimiento para comparar implementaciones secuenciales y paralelas con OpenMP. El objetivo es determinar cuándo la paralelización produce una mejora real, medir su escalabilidad y documentar los casos en los que el overhead, la jerarquía de memoria o las características del hardware limitan el rendimiento.

Contenido

- [Integrantes](#)
- [Problemas implementados](#)
- [Estructura del repositorio](#)
- [Requisitos](#)
- [Compilación](#)
- [Ejecución y correctitud](#)
- [Metodología del benchmark](#)
- [Resultados consolidados](#)
- [Conclusiones finales](#)
- [Reproducibilidad y limitaciones](#)

Integrantes

Integrante	Carné
Vianka Vanessa Castro Ordoñez	23201
Mia Alejandra Fuentes Mérida	23775
Jorge Luis Felipe Aguilar Portillo	23195

Problemas implementados

Se seleccionaron dos problemas con patrones de cómputo y acceso a memoria diferentes:

Problema	Implementación secuencial	Implementación paralela
Multiplicación de matrices densas	Recorrido <code>i-k-j</code> , que reutiliza <code>A[i][k]</code> y favorece la localidad de memoria.	División de <code>C</code> en bloques de <code>32 x 32</code> con <code>#pragma omp parallel for collapse(2) schedule(static)</code> . Cada hilo escribe en bloques exclusivos.
Filtro blur en escala de grises	Ventana deslizante de tres filas sobre una vecindad de <code>3 x 3</code> .	División por filas con <code>#pragma omp parallel for schedule(static)</code> . La entrada es compartida de solo lectura y cada hilo escribe filas distintas.

No se requieren regiones `critical`, operaciones `atomic` ni bloqueos: las iteraciones escriben en regiones independientes. Las entradas se generan de forma determinista y cada ejecutable imprime un checksum para

validar que las versiones secuencial y paralela produzcan el mismo resultado.

La explicación detallada se encuentra en:

- [Contexto, algoritmos y datos de prueba](#)
- [Estrategia de paralelización](#)

Estructura del repositorio

```
PartialParalelas_1/
├── secuencial/
│   ├── matrices_secuencial.c
│   └── blur_secuencial.c
├── paralelo/
│   ├── matrices_paralelo.c
│   └── blur_paralelo.c
├── docs/
│   ├── contexto_y_datos.md
│   ├── estrategia_paralelizacion.md
│   └── resultados/
│       ├── Mia_Fuentes/
│       ├── Vianka_Castro/
│       └── Jorge_Luis_Felipe_Aguilar_Portillo/
├── benchmark.ps1
├── generar_graficas.ps1
└── README.md
```

Cada carpeta individual de resultados contiene:

- **benchmark_raw.csv**: las mediciones individuales y sus checksums.
- **benchmark_summary.csv**: promedio, mediana, mínimo, máximo y desviación estándar.
- **benchmark_summary.md**: tabla compacta y datos del equipo utilizado.
- **resultados_metricas.md**: análisis completo de speedup y eficiencia.
- **graficas/**: curvas de speedup y eficiencia de ambos problemas.
- **evidencia_corridas.txt**: comandos y salidas representativas, cuando está disponible.

Requisitos

- Un compilador de C con soporte para OpenMP.
- Optimización **-O2** para reproducir las condiciones de las mediciones.
- PowerShell para ejecutar el benchmark automatizado y generar las gráficas.
- En Windows, GCC mediante MSYS2/MinGW-w64.
- En macOS, Apple Clang y **libomp** de Homebrew.

Compilación

Windows con GCC/MSYS2

Desde la raíz del repositorio:

```
$env:PATH = "C:\msys64\ucrt64\bin;$env:PATH"
gcc -O2 -fopenmp secuencial/matrices_secuencial.c -o matrices_secuencial.exe
gcc -O2 -fopenmp paralelo/matrices_paralelo.c -o matrices_paralelo.exe
gcc -O2 -fopenmp secuencial/blur_secuencial.c -o blur_secuencial.exe
gcc -O2 -fopenmp paralelo/blur_paralelo.c -o blur_paralelo.exe
```

macOS con Apple Clang y Homebrew

La contribución de Felipe comprobó el proyecto en Apple Silicon con Apple Clang 21 y `libomp` 22.1.8:

```
brew install libomp
OMP="$(brew --prefix libomp)"

clang -O2 -Xpreprocessor -fopenmp -I"$OMP/include"
secuencial/matrices_secuencial.c -L"$OMP/lib" -Wl,-rpath,"$OMP/lib" -lomp -o
matrices_secuencial
clang -O2 -Xpreprocessor -fopenmp -I"$OMP/include" paralelo/matrices_paralelo.c -
L"$OMP/lib" -Wl,-rpath,"$OMP/lib" -lomp -o matrices_paralelo
clang -O2 -Xpreprocessor -fopenmp -I"$OMP/include" secuencial/blur_secuencial.c -
L"$OMP/lib" -Wl,-rpath,"$OMP/lib" -lomp -o blur_secuencial
clang -O2 -Xpreprocessor -fopenmp -I"$OMP/include" paralelo/blur_paralelo.c -
L"$OMP/lib" -Wl,-rpath,"$OMP/lib" -lomp -o blur_paralelo
```

Ejecución y correctitud

Los programas aceptan los siguientes argumentos:

Ejecutable	Uso
<code>matrices_secuencial</code>	<code><tamaño></code>
<code>matrices_paralelo</code>	<code><tamaño> <hilos> [bloque]</code>
<code>blur_secuencial</code>	<code><ancho> <alto></code>
<code>blur_paralelo</code>	<code><ancho> <alto> <hilos></code>

Ejemplo de validación rápida en Windows:

```
.\matrices_secuencial.exe 512
.\matrices_paralelo.exe 512 4 32

.\blur_secuencial.exe 1920 1080
.\blur_paralelo.exe 1920 1080 4
```

Para cada problema y tamaño, el checksum secuencial debe coincidir exactamente con el paralelo. Esta comprobación se realizó satisfactoriamente en las **375 corridas medidas** de los tres integrantes.

Metodología del benchmark

Para mantener comparables las pruebas se usó el mismo protocolo en los tres equipos:

- 1. Una corrida de calentamiento por configuración, excluida de las estadísticas.
- 2. Cinco corridas medidas por configuración.
- 3. Matrices de 1024, 1536 y 2048 elementos por lado.
- 4. Imágenes de 3840 x 2160 (4K) y 7680 x 4320 (8K).
- 5. Versiones OpenMP con 1, 2, 4 y 8 hilos.
- 6. Bloques de 32 x 32 para la multiplicación paralela.
- 7. Medición exclusiva de la región de cálculo; se excluyeron reserva de memoria, generación de datos y checksum.

El benchmark completo en Windows se ejecuta con:

```
.\benchmark.ps1 -Integrante "Nombre Apellido"
```

Para una validación corta antes de lanzar todas las pruebas:

```
.\benchmark.ps1 -Integrante "Nombre Apellido" -Quick
```

Los archivos se guardan en docs/resultados/Nombre_Apellido/. A partir del resumen CSV se generan las cuatro gráficas con:

```
.\generar_graficas.ps1 -Integrante "Nombre_Apellido"
```

Las métricas se calcularon respecto a la versión secuencial real del mismo problema y tamaño:

Speedup(p)

Eficiencia(p)

Eficiencia (%)

= tiempo_secuencial / tiempo_paralelo(p)

= Speedup(p) / p

= Eficiencia(p) x 100

Resultados consolidados

Los siguientes valores corresponden a la configuración de 8 hilos, que obtuvo el menor tiempo paralelo en todos los tamaños probados. Las columnas de matrices muestran el speedup para 1024, 1536 y 2048; las de blur muestran 4K y 8K.

Equipo de prueba	Matrices 1024	Matrices 1536	Matrices 2048	Blur 4K	Blur 8K
Mia — Intel i5-1135G7, 4C/8T, Windows 10	0.996x	1.306x	1.471x	3.467x	3.388x

Equipo de prueba	Matrices 1024	Matrices 1536	Matrices 2048	Blur 4K	Blur 8K
Vianka — Intel i7-1165G7, 4C/8T, Windows 11	1.297x	2.250x	2.829x	3.518x	2.515x
Felipe — Apple M4 Pro, 14 núcleos, macOS	3.143x	3.977x	2.599x	5.862x	6.568x

Un valor menor que **1.0x** significa que la versión paralela fue más lenta que la secuencial. Por ello, la tabla muestra que el blur se benefició consistentemente de OpenMP, mientras que el resultado de matrices varió según el tamaño y el equipo.

Aporte de Felipe

Felipe añadió un tercer conjunto completo de resultados obtenido en una MacBook Pro con Apple M4 Pro, 24 GB de memoria y macOS arm64. Su contribución incluye 125 mediciones, estadísticas de dispersión, evidencia de compilación con Apple Clang/**libomp**, análisis y cuatro gráficas.

Sus resultados refuerzan las tendencias generales y amplían la comparación a una arquitectura distinta:

- En matrices, el mejor caso fue **1536 x 1536** con 8 hilos: **3.977x de speedup** y **49.71 % de eficiencia**.
- En blur 8K, 8 hilos redujeron el promedio de **0.074775 s** a **0.011385 s**: **6.568x de speedup** y **82.10 % de eficiencia**.
- En matrices **2048 x 2048**, 2 hilos todavía fueron más lentos que la versión secuencial; la mejora apareció a partir de 4 hilos.
- En todas sus configuraciones, los checksums secuenciales y paralelos coincidieron.

Resultados detallados

- [Resultados de Mia Fuentes](#)
- [Resumen actualizado de Vianka Castro](#)
- [Resultados de Jorge Luis Felipe Aguilar](#)

Conclusiones finales

1. **El filtro blur es el candidato más sólido para OpenMP.** En los tres equipos obtuvo aceleraciones de entre **2.515x** y **6.568x** con 8 hilos. La división por filas ofrece trabajo uniforme, no exige sincronización durante el cálculo y conserva buena localidad de acceso.
2. **La multiplicación de matrices no mejora de manera universal.** Aunque la versión de 8 hilos alcanzó hasta **3.977x**, también hubo un caso de **0.996x**, sin mejora respecto a la versión secuencial. El costo de organizar el trabajo por bloques, la caché y el ancho de banda de memoria pueden impedir que el paralelismo compense el trabajo adicional.
3. **Más hilos redujeron el tiempo paralelo, pero no mantuvieron eficiencia lineal.** En general, la eficiencia descendió al aumentar el número de hilos debido al overhead de OpenMP, la competencia por memoria y, en los equipos Intel, el uso de 8 hilos lógicos sobre 4 núcleos físicos.

4. **El tamaño del problema determina si conviene paralelizar.** Las regiones de cálculo cortas son más sensibles al overhead y al ruido de medición. La paralelización debe aplicarse cuando la carga por hilo sea suficiente para amortizar esos costos.
5. **El hardware y la plataforma cambian el punto de equilibrio.** Los resultados del M4 Pro fueron más favorables, pero los tiempos absolutos no deben compararse como si solo variara el procesador: también cambiaron el sistema operativo, el compilador y el runtime de OpenMP.

Como recomendación de consultoría, el blur puede utilizar la versión OpenMP por defecto para imágenes grandes. En matrices conviene medir primero en el equipo objetivo y ajustar tamaño de bloque, recorrido y cantidad de hilos antes de sustituir la versión secuencial.

Reproducibilidad y limitaciones

- Las mediciones comparables son las realizadas **dentro de un mismo equipo**, siempre contra su propia línea base secuencial.
- Cinco repeticiones permiten observar variación básica, pero no sustituyen un estudio estadístico prolongado.
- La versión paralela de matrices usa un recorrido por bloques diferente al **i-k-j** secuencial. Por eso, OpenMP con un hilo no aísla únicamente el costo de crear o administrar hilos.
- Los tiempos muy cortos, especialmente en blur, pueden verse afectados por caché, frecuencia turbo, temperatura y procesos del sistema.
- Las gráficas, CSV crudos y desviaciones estándar se conservaron para que los resultados puedan auditarse y repetirse.

Repositorio

- GitHub: [FelipeAP04/ParcialParalelas_1](https://github.com/FelipeAP04/ParcialParalelas_1)
- Clonar: `git clone https://github.com/FelipeAP04/ParcialParalelas_1.git`